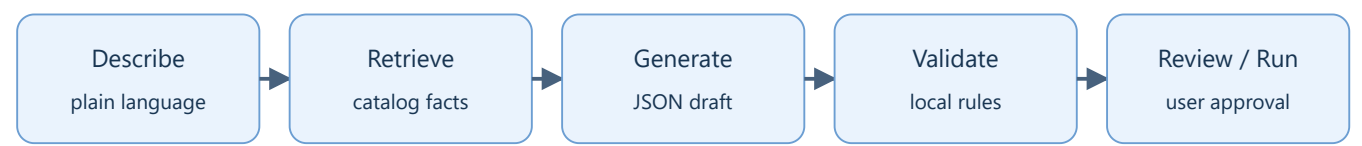


How AI works inside TrialWyze Rest Tester

Four workflows. One grounded planning engine.
Human approval at every execution boundary.

AI designs a draft; deterministic code checks it; the user decides



The simplest explanation: AI is a test-design assistant. It writes a recipe from the application's API cookbook. Ordinary Python checks the recipe. You decide when to cook it.

Audience	First-time AI users, testers, developers and people demonstrating the application
Implementation snapshot	3 October 2026; current local working tree, including uncommitted AI changes
Scope	Ask AI, API Explorer, Test Suites, CSV Data Runner, Load Testing, Settings, Activity, token usage and notifications
Evidence	Local code inspection and focused offline regressions. No live provider calls for this document.

This is an implementation guide, not the earlier AI roadmap. Examples are illustrative unless labeled as implementation excerpts. Endpoint IDs, field names and routes must come from the loaded catalog. No personal credentials, saved prompts or CSV row data are included.

Reading map and teaching guide

You can explain the system without starting with Python. Read Chapters 1-4 first, walk through one prompt, then introduce the different workflows. Developers can use the component tables and generated function index.

A five-minute explanation

1. **Choose a workflow.** Is this one API call, several dependent calls, CSV rows, or many simulated users?
2. **Describe the goal.** "Get brand 1 and design a small load scenario" is clearer than "test everything."
3. **The app finds relevant API facts locally.** It sends selected cards, not the entire workspace.
4. **AI proposes JSON.** Python checks endpoints, fields, types, dependencies and supported features.
5. **Review and open the draft.** Opening does not send the API request.
6. **Execute separately.** The normal editor and runner still own validation, approvals, HTTP calls and results.

Important honesty: "Validated" means the proposal fits the app's known rules. It does not prove the API will succeed, that test data exists, or that a provider will always finish within its token limit.

Terminology convention: "token budget" below means a generation ceiling or consumption count, not a bank balance. "Plan ready" means a validated draft, not a completed API test. "Thinking" in Activity means observable stages, not access to private model reasoning.

1. What AI does - and does not do

Rest Tester is a TrialWyze-specific desktop application. Its catalog knows the platform's services, routes, parameters, filterable properties, sortable properties and payload schemas. AI sits on top of these facts and translates a natural-language goal into an editable test draft.

AI is responsible for

- Interpreting a testing goal.
- Choosing useful endpoints from supplied candidates.
- Suggesting parameters, filters, sorting and payloads.
- Designing suites, CSV mappings or load stages.
- Explaining assumptions, risks and clarification questions.

AI is not responsible for

- Automatically running TrialWyze requests.
- Granting load or write permission.
- Discovering live database records.
- Managing your API authentication.
- Replacing native assertions or execution engines.
- Guaranteeing correctness or estimating cloud cost.

Analogy: a junior test engineer plus a strict reviewer. The assistant proposes a test; the reviewer checks the contract; the team lead approves execution. A beautifully written test can still fail against real data. The reviewer cannot infer the state of a server that has not been called.

The four outputs

Ask AI mode	Produced draft	Where the user executes
Single request	One endpoint with parameters, filters, sort and body	API Explorer
Test Suite	Ordered cases, assertions, captures and dependencies	Test Suites
Data Runner	One request template plus CSV column mappings/transforms	CSV Data Runner
Load Testing	One target plus virtual-user stages and thresholds	Load Testing Studio

Why this is not a generic chatbot

The prompt contains catalog cards for actual endpoints. The validator uses the same catalog and platform helpers. For example, a request to sort Brands is not allowed to silently use a Products-only property. The AI describes a filter structurally; the app's encoder produces TrialWyze's actual wire grammar.

There is no fine-tuning pipeline, embedding database, autonomous tool-execution agent or cross-service server dependency in this AI layer. Its intelligence comes from local retrieval, carefully constrained prompts, typed plan objects, deterministic checks and a human-controlled handoff.

Source anchors: `api_tester/ai/prompts.py`; `request_plan.py`; `planner.py`; `workflow_plan.py`; `main.py`.

2. The screen and user journey

The Ask AI window follows a compact, plan-first layout. It intentionally avoids turning into a second API editor. The result occupies the main area; the composer at the bottom starts a new planning operation.

Control	What it means	Beginner explanation
Yellow bulb and warning	AI can make mistakes; nothing runs without approval	A reminder, not an error
Plan area	Readable result, assumptions, warnings and problems	The proposed recipe
Copy plan JSON	Copies the structured draft, when available	A machine-readable copy
Open in destination	Transfers the reviewed draft to the relevant native tool	Put the recipe on the workbench
Mode selector	Selects request, suite, CSV or load planning	Choose the kind of test
Choose CSV	Available for Data Runner; reads column names	Tell AI the spreadsheet's headings
Prompt box	Goal written by the user	Your testing instruction
Provider/model label	Read-only active connection; single-line with elision	Which assistant you are asking
Grey/blue dot	Connection availability status	Unavailable / available
Settings gear	Opens AI Settings; disabled during generation	Change the assistant's connection
Cancel / Ask	Cooperative cancellation / begin generation	Stop waiting / request a proposal
Activity / Token details	Execution diagnostics / per-call consumption	What happened / what was consumed

Useful prompt pattern

```
Mode: Load Testing
Goal: Load test the GET brand-by-id endpoint.
Input: Use brand ID 1.
Workload: Use a small steady stage; explain assumptions.
Safety: Do not run anything.
```

A user-supplied ID is a constant, not proof that the record exists. Exact endpoint names vary with the catalog. The destination editor still requires the correct service base URL and API credentials.

During generation, the mode and editing controls are protected from concurrent changes. Completed results can be reopened through notifications. A historical result is a snapshot, not an invitation to rerun the model silently. Long provider names have a full tooltip rather than pushing the controls out of the window.

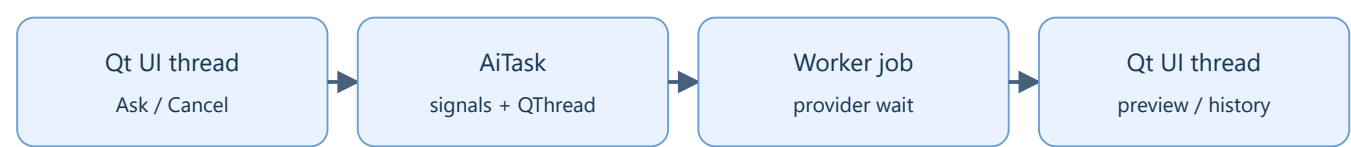
Source anchors: AskAiDialog._build, ask, _set_busy, restore_result; MainWindow._show_ask_ai.

3. The architecture in plain language

Each component has a narrow job. This keeps cloud-specific code away from UI behavior and keeps test execution away from model output. Think of an office with a receptionist, librarian, writer, reviewer and filing clerk.

Component / source	Job	Analogy
ui.py / AskAiDialog	Collect goal, show preview, offer handoff and diagnostics	Reception desk
controller.py / AiTask, _Worker	Run slow work off the UI thread; send Qt signals	Job dispatcher
catalog_index.py / CatalogIndex	Rank endpoints and prepare compact cards	Librarian
prompts.py	Versioned instructions and repair templates	Writing brief
planner.py	Coordinate search, provider calls, checks and limited repair	Test-design supervisor
request_plan.py / suite_plan.py / workflow_plan.py	Represent and convert structured proposals	Standard forms
validation.py and workflow/suite validators	Check catalog rules and supported behavior	Contract reviewer
provider.py + providers/	Common interface with hosted/Ollama adapters	Translator for each supplier
config.py / redaction.py	Profiles, key storage, outbound masking	Configuration cabinet and privacy filter
usage_store.py / usage_ui.py / usage_dashboard.py	Record and display call metadata	Meter and ledger
main.py + native tool UIs	Route approved drafts into normal editors	Workbenches
runner.py / native data/load engines	Execute tests only after the normal user action	Workshop machinery

The UI stays responsive while a worker waits

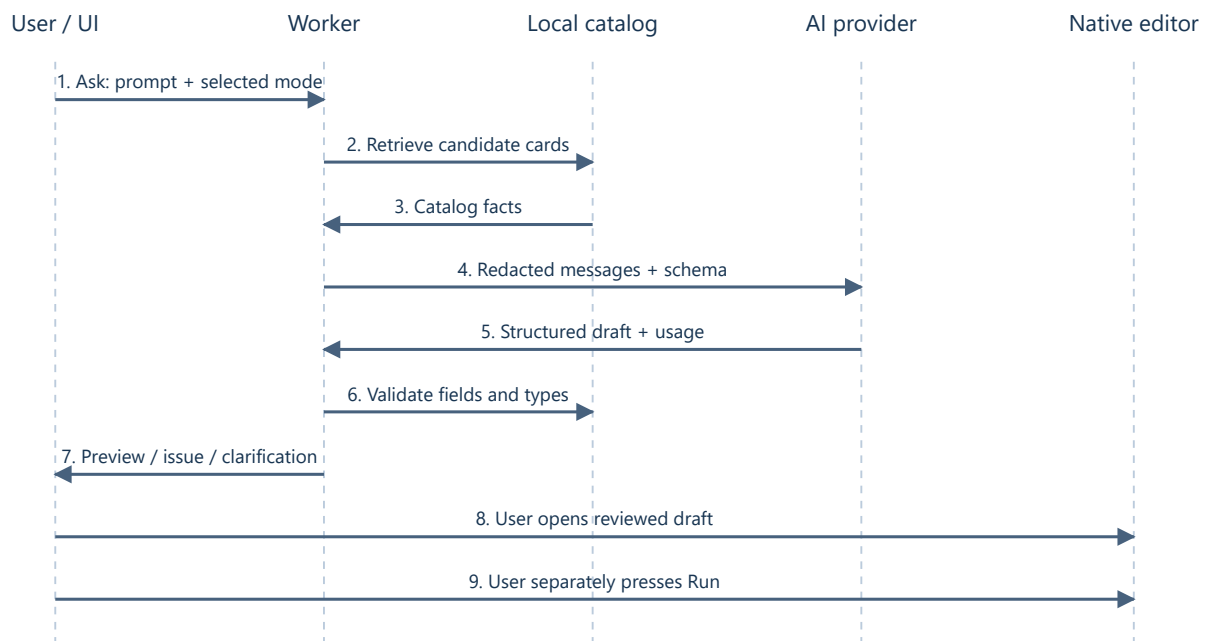


What is reused?

WorkflowPlanner inherits RequestPlanner. It reuses search, cancellation, usage and repair flow, but overrides the system prompt, response schema, plan parser, validator and output-token ceiling. SuitePlanner also shares helper behavior but has its own loop because it designs multiple cases.

Conversion uses the app's existing filter/sort encoders and native model classes. This is important: the AI does not create a second incompatible test format or duplicate the load engine.

4. One complete request: sequence diagram



Read the arrows like a conversation

1-3: The user presses Ask. A worker searches the local catalog and collects useful endpoint summaries/cards. No microservice API call is needed for retrieval.

4-5: The provider receives redacted instructions, selected API facts and an output schema. A hosted provider requires cloud consent and its own API key. It returns a proposed plan plus usage metrics when available.

6-7: Python parses the output and checks it against the catalog. The UI receives a plan, clarification or unsupported result. An ordinary error may trigger a limited repair call; token truncation ends the generation.

8-9: The user opens a draft in a native editor and later presses Send/Run/Start. These are separate actions. The AI worker never calls the TrialWyze API execution engine.

Analogy: ordering a custom cake. The designer proposes ingredients and steps from the bakery's menu. The kitchen checks what equipment and ingredients are allowed. Receiving the drawing does not start the oven; the customer must approve the order.

Separate failure boundaries

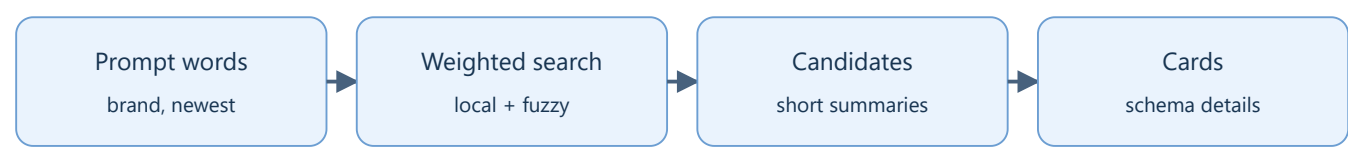
- **Provider failure:** bad key, network timeout, quota or unavailable model.
- **Output failure:** malformed JSON, wrong object shape or token-limit truncation.
- **Plan failure:** unknown parameter, invalid operator, dependency issue or unsupported load stage.
- **Runtime API failure:** only after user execution; missing record, API rejection or failed assertion.

It is misleading to describe every failure as "AI is disconnected." The connection probe, plan-generation call, catalog validator and native API runner answer different questions. A successful probe is not a guarantee that a large suite fits within its output budget.

Sequence arrows are conceptual; actual Qt signal delivery and repair rounds are described in Chapters 11-12.

5. The catalog and local retrieval

A librarian selects useful pages instead of sending the whole library



The catalog is a local snapshot of API contracts. It describes services, endpoint IDs, method/path, parameter sources, filter metadata, payloads and available response schemas. It is refreshed on demand; a microservice change does not automatically retrain AI or silently regenerate the test application's catalog.

How search works

- 1. `split_words` splits punctuation and CamelCase names.
- 2. `singular` normalizes common plural forms; stopwords are removed from the query.
- 3. `_document` assigns weights: controller 3, path/action 2, service 1, parameter/payload names 0.5.
- 4. Inverse document frequency gives distinctive words more value than ubiquitous words like "id".
- 5. `SequenceMatcher` gives a fuzzy bonus for nearby words at similarity 0.85 or above.
- 6. Method words nudge ranking: "create" toward POST, "count" toward HEAD, "list" toward GET.
- 7. Single-record cues and list cues nudge GET-by-ID versus GET-list. Results are sorted deterministically.

Important: These search "tokens" are words used for ranking. They are not the model's billable subword tokens. No embedding service or vector database is involved.

Small context instead of the entire workspace

Workflow	Retrieval behavior
Request / Data / Load	Defaults: 8 candidate summaries and 3 detailed cards; settings can change these
Suite	Search plus siblings of strongest matching controllers; up to 24 offered endpoints, up to 10 cards
Endpoint cards	Parameters, supported filter operators, sortable fields, payload and expected status
Suite cards	Also include response information for assertions/captures; filter-field count is reduced

Request cards default to at most 40 filter fields and 12 LOV values per list; suite cards use at most 20 filter fields. These are context-reduction controls, not measured model-token caps. If AI selects an endpoint shown only as a summary, the planner supplies its full card and asks for refinement.

Coverage limitation: missing response schemas may be supplemented with likely record fields from sibling filter schemas. Those are guesses, not authoritative response contracts; review assertions.

6. Prompts, schemas and typed plans

A **prompt** is an instruction packet, not just the user's sentence. A **schema** describes the expected JSON shape. A **plan object** is the Python representation after parsing. These layers are related but not interchangeable.

Layer	Contents	Why it exists
System prompt	Rules: never execute, use shown fields, write assumptions, ask clarifications	Tell the model its job
User packet	User goal + ranked endpoint summaries + selected cards	Ground the answer in API facts
Response schema	Properties, allowed candidate IDs and required structure	Request machine-readable output
Parsed plan	RequestPlan, SuitePlan or WorkflowPlan	Give application code a known shape
Validator	Actual catalog checks, transforms, types and dependencies	Check meaning, not only syntax

Three possible statuses

- **plan**: proposes a draft for review.
- **clarify**: asks one question with short choices; answering can generate a revised plan.
- **unsupported**: the offered contracts/features cannot represent the goal.

Illustrative request JSON

```
{
  "status": "plan",
  "title": "Brands whose name contains corn",
  "endpoint_id": "<ID from the loaded catalog>",
  "parameters": [{"name": "PageSize", "value": "10"}],
  "filters": [{"field": "Name", "op": "ct", "values": ["corn"]}],
  "sort": [{"field": "CreatedDate", "descending": true}],
  "payload": null,
  "expected_status": "200-299",
  "assumptions": ["Newest means descending creation date."],
  "warnings": []
}
```

The field names above are illustrative. They are only usable when the selected endpoint card lists them. The model should not manufacture a route or guess that every endpoint supports PageSize.

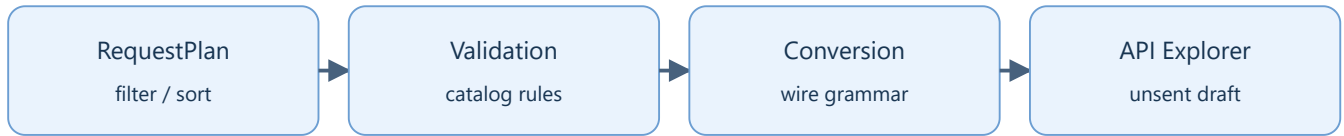
Provider differences

Hosted OpenAI/Azure/Sarvam calls request JSON object mode and include the schema in a system message. This is not full server-side enforcement of every application rule. Claude uses a forced `submit_plan` tool with an input schema; the tool output is read as data, not executed. Ollama uses its native `format` schema. Local parsing and catalog validation are still necessary for every provider.

Prompt versions currently identify request, suite and workflow instruction sets. Changing instructions changes behavior and should be followed by focused regression tests.

7. Single request / API Explorer

Single request: a recipe is not a cooked meal



A single-request plan is the smallest workflow. It selects one endpoint and specifies constants for path/query parameters, structured filters/sorting, an optional payload and expected HTTP status. It does not create test data on the server.

Example: read a brand

1. User says: "Get brand 1."
2. Local retrieval favors a GET-by-ID endpoint for the Brand controller.
3. AI returns the catalog endpoint ID and the exact path-parameter name/value.
4. The validator checks the ID and parameter type. A missing path value produces a warning/refinement.
5. `to_explorer_request` converts the checked plan into the editor's value map.
6. `MainWindow.open_ai_request` loads it as an editable, unsent API Explorer draft.
7. User checks environment/base URL/authentication and presses Send.

Why filter encoding is not delegated to AI

TrialWyze filters use a specific grammar: `Name__op:=value`, with semicolon for AND and vertical bar for OR. Sorting uses comma-separated fields and a trailing minus for descending. The model returns structured `FilterSpec` / `SortSpec` objects; app helpers produce the wire text.

```
# Conceptual example: structured intent -> platform grammar
FilterSpec(field="Name", op="ct", values=("corn",))
SortSpec(field="CreatedDate", descending=True)

# Illustrative encoded query values, if these fields exist:
Filter = "Name__ct:=corn"
Sort = "CreatedDate-"
```

Analogy: address form versus mailing-label printer. AI fills named fields on the form. A deterministic printer formats the postal label. Letting AI invent punctuation would create avoidable errors.

HTTP semantics and native controls

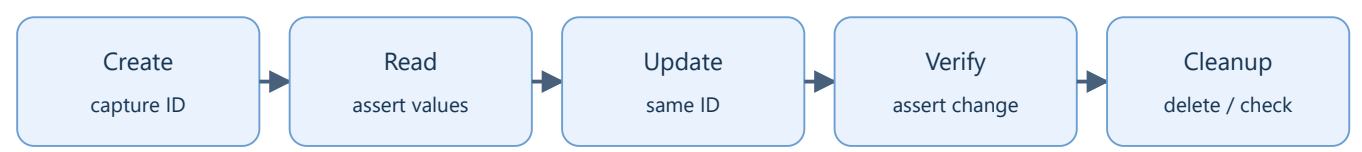
GET/HEAD/OPTIONS bodies are dropped by request validation and recorded as assumptions. Write methods receive risk warnings. PATCH follows RFC 6902 operation-array semantics rather than an arbitrary partial entity. Expected status is validated independently from assertions.

Native seeding/query builders remain available in API Explorer. AI suggestions do not replace them or fetch real IDs. Routing is taken from the catalog; service routes generally have no intrinsic `/api` prefix; APIM may add it in the environment base URL.

Core methods: `RequestPlanner.plan`; `validate_request_plan`; `to_explorer_request`; `MainWindow.open_ai_request`.

8. Test Suites and data dependencies

Lifecycle example - all IDs and fields must match the current catalog



A suite is an ordered collection of cases. Each case resembles a single request, with extra assertions, captured variables, dependencies and execution phase. AI can design a lifecycle because retrieval offers sibling endpoints on the same controller instead of only independent top search hits.

How a created ID reaches later cases

```
{
  "key": "create",
  "captures": [{"name": "brandId", "path": "$.Id", "source": "body"}]
}

{
  "key": "read",
  "parameters": [{"name": "id", "value": "{{brandId}}"}],
  "depends_on": ["create"]
}
```

This is an illustrative fragment, not a complete runnable suite. The native runner resolves variables after capturing them from actual responses. The app checks dependency references and rejects self-dependencies and invalid variable usage. It checks response paths when response schema coverage permits.

Suite concept	Easy meaning	Implementation concern
Case key	A short name used by other cases	Must be unique
Assertion	A check on the response	Known kinds, required path/value, compatible types
Capture	Save a response value for later	Valid variable name and body/header/status source
depends_on	This case needs another to complete	No missing, cyclic or invalid cleanup relationship
Cleanup / always_run	Attempt to remove test artifacts	Does not guarantee deletion succeeds
stop_on_failure	Stop normal dependent work when a case fails	Native runner decides execution order and skips

Assertion values are literal in the existing suite contract; AI must not insert `{{variables}}` into assertion values expecting substitution. Status expectations belong in `expected_status`. A baseline comparison needs a previously recorded native baseline, not an invented AI reference response.

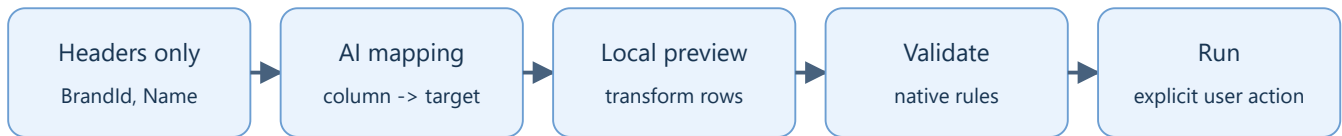
Where runner.py fits

`run_case` sends a native request and evaluates checks. `run_suite` manages dependencies, normal/cleanup phases and aggregate results. `export_report` produces a test report. None of these functions is the AI model-call path. Opening an AI suite loads an unsaved draft; the user runs it separately.

Existing cases trigger Replace/Append/Cancel choices. Appending preserves suite variables according to the existing handoff logic, so users should review naming collisions and dependencies before executing combined suites.

9. CSV Data Runner

CSV data stays local; AI receives column names only



The Data Runner applies one request template to multiple locally read CSV rows. AI designs how columns map to API inputs; it does not read the actual row values to design the mapping.

What is sent to the model?

The available column names, the user's goal, selected endpoint cards and the plan schema. The app does not send the CSV file path/name or row values through this workflow. Be aware that column names themselves can contain sensitive information, and anything manually typed in the prompt is subject to best-effort redaction.

```
# Illustrative CSV headings, not transmitted row content:
BrandId, NewName, ExpectedStatus

# A planned mapping:
{
  "column": "BrandId",
  "target_key": "path:Id",
  "transforms": [{"kind": "trim", "options": {}},
                 {"kind": "to_integer", "options": {}}]
}
```

Mappings and transforms

Allowed target forms include path/query/form fields, payload fields, supported filter operators, sorting, expected status, correlation key and skip-row behavior. Headers, credentials, secrets and file-path destinations are not AI mapping targets. Transforms come from the native closed list: examples include trim, required, default, case conversion, numeric conversion, date formatting and prefix/suffix. Supported kinds and options are checked, not accepted as arbitrary executable Python.

Analogy: a mail-merge template. AI connects spreadsheet headings to blanks in a letter. The local mail-merge engine later reads each row and fills the blanks. AI never needs the mailing list itself.

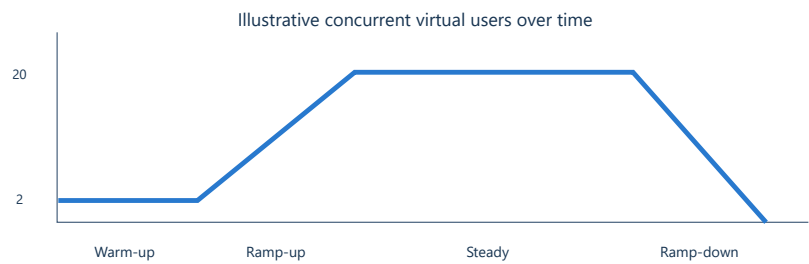
Handoff defenses

1. The main window revalidates against the current catalog.
2. The user confirms replacement of the native Data Runner draft.
3. `load_ai_draft` rereads headers; changed columns reject the old plan.
4. The native UI loads source settings, local preview and supported mappings.
5. Any active run/validation blocks replacement.
6. Run remains disabled until the native validation step is completed.

A mapped required path parameter is not filled with a fabricated constant. Payload fields supplied by mappings are treated differently from genuinely missing payload constants. Request constants are not allowed to contain unresolved CSV placeholders; mappings are the proper channel for row data.

Seeding distinction: the app's native data-generation/seed facilities are separate from AI mapping design. AI does not automatically seed a database or execute a creation API.

10. Load Testing Studio



AI proposes one request target and a supported closed-virtual-user schedule. A closed model keeps a number of virtual users active; each user sends requests and may pause for a configured think time. It is not a promise to deliver an exact arrival rate such as 1,000 requests per second.

Supported stage	Meaning
warm_up	Prepare the service/client at low load
ramp_up	Increase active virtual users
steady	Hold the selected population
ramp_down	Reduce load toward completion

Illustrative stage and threshold

```
{
  "stages": [{
    "kind": "steady", "duration_seconds": 30,
    "start_users": 5, "end_users": 5, "think_time_ms": 200,
    "counts_toward_sla": null, "label": "Small read test"
  }],
  "thresholds": [{"metric": "p95_ms", "operator": "<=",
    "target": 500, "label": "Latency target"}]
}
```

This example is design input, not a capacity guarantee. Five users can generate very different throughput on fast versus slow APIs. A p95 limit means 95% of the measured samples should be at or below the target, subject to the native engine's metric and stage-inclusion rules.

Strict representability

The AI contract accepts only supported stage/threshold types and exact number types. Durations must be finite, 0.1-86,400 seconds, with at most one decimal place; total duration is bounded. Users are integer values up to 100,000; think time is integer milliseconds up to 600,000. Thresholds have numeric bounds/precision and error-rate targets are fractions: 0.01 means 1%, not 0.01%.

These parser bounds are not recommended operational loads. Start tiny and use an approved non-production environment. Open-rate workloads, unsupported stage kinds, multiple targets and unresolved variables are not silently approximated; the planner should clarify or report unsupported.

Permissions are cleared, not inherited

Opening an AI draft clears environment permission, write confirmation and pre-run approval. Start stays disabled until the native summary/validation and safety controls are satisfied. Existing active load execution blocks draft replacement. AI never starts the engine or checks an approval checkbox on your behalf.

11. Validation and repair

JSON validity is like spelling correctness; catalog validity is like factual correctness. A syntactically valid object can still name a nonexistent filter, an unsupported transform or a dangerous workload type.

What deterministic validation checks

- Endpoint exists in the catalog; parameters exist and have compatible scalar/enum values.
- Filter and sort fields are allowed; operators and value counts match field metadata.
- Payload fields match available schema rules; PATCH uses supported RFC 6902 operation structure.
- Suite cases have valid keys, assertions, capture paths, variables and dependency relationships.
- CSV column names and destinations exist; transformations are recognized and safe to represent.
- Load numbers are finite, correctly typed, bounded and supported by the native UI contract.

Small corrections versus real errors

The validator can canonicalize case differences and operator aliases, move raw filter text into structured filters, drop bodies from read operations and normalize supported values. These adjustments are recorded as assumptions. Unknown fields or meaningful incompatibilities become `PlanIssue(path, code, message)`.

```
# Conceptual result:
PlanIssue(
  path="filters[0].field",
  code="unknown_filter",
  message="Use a filterable property from this endpoint's card."
)

# Planner sends a repair instruction quoting valid options.
# It does not send the TrialWyzé request to discover if the field works.
```

Analogy: an expense form. The clerk can correct capitalization, but cannot approve an unknown project code. That form must go back to the author with a precise problem statement.

Why multiple model calls happen

One initial call is followed by up to `max_repairs` repair/refinement calls. A call may be needed because JSON failed, a plan violates the catalog, the selected endpoint needs its detailed card, or a path value requires refinement. Refinement does not always mean the model made a mistake; sometimes it needs more facts.

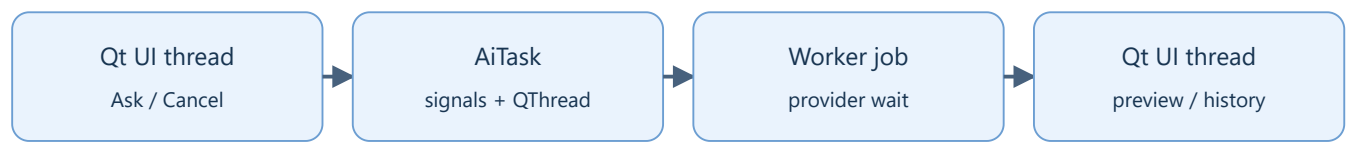
Exception: hosted structured output marked `length` or `max_tokens` is not repaired with an identical budget. It stops immediately. No partial JSON concatenation, synthetic closing braces or silent success fallback is used.

Prompts discourage invented fields, but they are not a security boundary on their own. Validators are the local check layer; they also have catalog-coverage limitations and do not prove server-side business behavior.

12. Threads, progress and cancellation

A model call can take seconds or minutes. If it ran in the Qt UI thread, the window would freeze. The controller uses a worker QObject moved to a QThread and relays result/error/progress signals back to the UI.

The UI stays responsive while a worker waits



Core signal responsibilities

Signal / method	Meaning
AiTask.start	Starts the background thread
progress(str)	Observable planning stage/diagnostic
succeeded(object)	Job returned an outcome/report; inspect its ok/status
failed(str)	Cancelled, provider failure or surfaced unexpected exception
done()	Thread completed; UI can restore controls and save a snapshot
AiTask.cancel	Sets a threading.Event; it is not an immediate HTTP abort

Why Cancel can take time

Cancellation is cooperative. The planner checks the flag before calls and at planning checkpoints. An in-flight non-streaming HTTP request may finish or time out before the worker can observe cancellation. The worker discards the result when the cancellation flag is set. An HTTP request that already reached the provider may still be billed; discarding its answer does not undo generation.

Analogy: asking a colleague to stop an errand. You can leave a stop message, but if they are already at the counter they may return before reading it. You cannot rewind a purchase already made.

Busy indicators

The header AI button rotates an activity arc during Ask AI execution, not during every Settings connection test. The connection dot indicates availability, not generation progress. These indicators intentionally answer different questions. Closing the reusable Ask AI dialog hides it; it does not automatically cancel a background generation.

Non-streaming is intentional in the current implementation

Ollama explicitly requests `stream=False`; hosted calls use complete JSON HTTP responses rather than an SSE renderer. Activity shows real application stages and completion diagnostics, but it does not stream partial plan tokens. A visible timer is not evidence that the model is producing a usable final answer.

Source anchors: `_Worker.run`; `AiTask`; `plan_request_task`; `plan_suite_task`; `plan_workflow_task`; `AskAiDialog_cancel`; `HeaderAiButton.set_busy`.

13. Providers and connection probes

The provider interface is the common contract. Adapters translate it to different APIs. A provider key authenticates the model call; it is not the TrialWyze access token. Hosted adapters require HTTPS and reject redirects.

Provider	Transport / authentication	Structured output
Ollama	Native /api/chat, typically localhost	format = schema; think; options.num_predict/num_ctx
OpenAI	/chat/completions; Bearer key	JSON object mode + schema instruction; max_completion_tokens
Azure OpenAI	/openai/deployments/{deployment}/chat/completions + api-version; api-key	Same JSON mode; deployment name may not reveal model family
Claude	/v1/messages; x-api-key + anthropic-version	Forced submit_plan tool input; max_tokens
Sarvam	/v1/chat/completions; api-subscription-key	JSON mode; max_tokens; reasoning_effort=null for structured plans

Probe policies

Provider	Connection probe	What it proves
Ollama	GET /api/tags; confirm configured name or :latest; no generation	Server/model-list availability, not inference quality
Sarvam	"Reply with OK"; up to 128 tokens; minimum 30 seconds; reasoning disabled	The selected provider/model returns final text
OpenAI / Azure / Claude	One probe; up to 4,096 output tokens; minimum 60 seconds; no forced temperature/reasoning	A successful final-text response with model defaults

Probe limits are application policies, not provider-prescribed numbers. A probe copy prevents probe-only settings from mutating planning behavior. Longer explicitly requested timeouts are preserved. Hosted startup probes are billed for actual usage and are recorded; there is no automatic probe retry.

Failure classes distinguish authentication, rate/quota, timeout, missing model/endpoint, HTTP rejection and response-format errors. Empty/whitespace final text fails with safe stop/filter/truncation details. Arbitrary server bodies are not shown because they can echo sensitive data.

Model compatibility still matters. The normal planning adapter is not a universal capability-discovery engine. Some model families reject normal sampling/schema/tool settings or need another API. Use a compatible chat model and verify provider documentation; a successful probe alone is insufficient.

14. Settings and secret storage

AI Settings manages named connections, much like environments. Multiple accounts/models for one provider can coexist, but only one connection is active. The table has Use, Connection, Provider, Model/Endpoint and Actions.

Setting	Scope / meaning
Connection name / provider / endpoint / model	Identify the selected model service
API version	Azure request version; default 2024-10-21 in current code
Hosted consent	Explicit permission before sending to a hosted model
Key / environment-variable binding	Credential storage or external resolution
Ollama think / context window	Local reasoning flag and requested context size
Timeout / repair rounds	Shared planning controls; loaded values are bounded
candidate_count / detailed_candidates	Retrieval controls in configuration, not tokenizer settings

Defaults are not necessarily your active settings

Code defaults include 600 seconds per planning call, 2 repairs, 8 candidates, 3 cards and Ollama context 16,384. Loading clamps timeout to 10-1,800 seconds, repairs to 0-5, candidates to 1-20 and context to 2,048-262,144. These are configuration bounds; the chosen model may support a smaller context.

Where credentials live

Settings JSON stores connection metadata, not plaintext provider keys. Keys use `msal_extensions.build_encrypted_persistence` in the AI credentials directory. A configured environment variable can supply a key. Each named connection has a credential ID; IDs are validated before constructing storage paths. Failed encrypted reads are surfaced instead of replaced with an empty success-shaped value.

```
# Environment bindings recognized by the app:
AZURE_OPENAI_API_KEY
OPENAI_API_KEY
ANTHROPIC_API_KEY
SARVAM_API_KEY
```

Analogy: address book plus locked key cabinet. The address book can say "Sarvam account, model X" without containing the key. The encrypted cabinet or environment provides the credential when a call is made.

The inline gear opens the same AI Settings workflow, not a new independent configuration screen. A deleted connection does not erase past usage records. The disabled active-profile Delete action remains visibly red but cannot delete the active connection until another connection is selected.

15. Tokenization explained correctly

A model does not read text in words or characters exactly as a person does. Its tokenizer converts text into integer IDs representing pieces of text. A word can be one token, several tokens, or share pieces with punctuation. Different providers/models can use different tokenizers.

Analogy: packing a suitcase. You do not pack "one outfit" as a single item; you pack shirts, shoes and socks. Tokens are the pieces occupying space. JSON punctuation and long field names occupy space too. A bigger instruction packet uses more pieces.

Two meanings of "token" in this app

Term	Where used	Meaning
Search token	CatalogIndex.tokens/query_tokens	Normalized words used for local endpoint ranking
Model token	Provider usage, output ceiling, context	Provider/model-specific encoded text units
Access token	TrialWyze API authentication	A credential; entirely unrelated to token consumption

What tokenizer settings are used?

No local model tokenizer is selected or configured. The AI layer does not use tiktoken, estimate exact input tokens before sending, or calculate model-specific context headroom. It controls context size by choosing a limited number of summaries/cards and controls generation with provider-native output limits.

The provider tokenizes the full request. That includes system instructions, user goal, cards, schema text, repair messages and any provider-specific wrappers. Word count, character count and the number of catalog fields are not exact input-token counts.

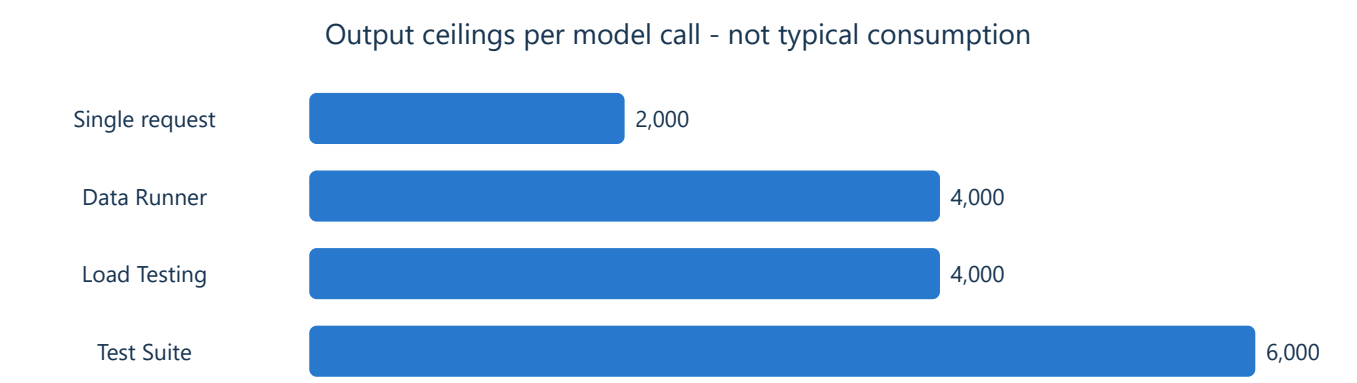
Usage types

- **Input tokens:** prompt/context processed by a model call.
- **Output tokens:** generated units as reported by the provider; reasoning may be included depending on provider semantics.
- **Reasoning tokens:** internal work on supported models; not equivalent to visible JSON.
- **Context window:** the model's working space, typically shared by input and generation under provider-specific rules.

A handy rough estimate such as "four English characters per token" is only a teaching aid. It must not be used as a guarantee for multilingual text, IDs, nested JSON or provider billing. This implementation uses reported metrics, not invented tokenizer precision.

Illustration: a 4,000-token ceiling does not mean the model must generate 4,000 tokens. A plan finishing after 450 generated tokens consumes the reported 450, subject to the provider's accounting semantics.

16. Token controls and context windows



Exactly which settings go on the wire?

Provider	Output control	Context/reasoning control
Ollama	options.num_predict = workflow ceiling	options.num_ctx from context_window; think from local profile; keep_alive="30m"
OpenAI / Azure	max_completion_tokens = workflow ceiling	No explicit context-window field in this adapter; model/service owns it
Claude	max_tokens = workflow ceiling	No configured thinking budget in normal planner requests
Sarvam	max_tokens = workflow ceiling	reasoning_effort=null for schema-based plans and probes

Normal planner calls request temperature 0.0 where the adapter currently sends it. Temperature changes variation, not tokenization or context capacity. Connection probes omit temperature for compatibility. Unsupported model-specific parameters must not be confused with a token-budget failure.

Why context and output are different

The input can contain thousands of tokens even when the final plan is short. A suite's cards and response schemas make its prompt larger than a single request. Repair rounds append additional instructions and, where appropriate, earlier responses, so later calls may process more input. The current app does not prune messages against an exact model-token count.

Analogy: desk size versus writing allowance. The context window is the desk that holds notes and writing. The output cap is the maximum pages you are allowed to write on this assignment. Enlarging the output allowance does not automatically enlarge the desk.

Ollama context correction

The local default context was made 16,384 to reduce suite-prompt truncation associated with smaller default windows. Profile settings may request another size. This is a request to the model server, not proof the hardware, model configuration or runtime honored that exact capacity. A larger context can also require more RAM/VRAM and increase latency.

No hidden autoscaling: the app does not dynamically raise a workflow ceiling, switch models, buy credits, or continue a cut-off answer. It uses the explicit policies above and reports failures.

17. Budgets, cost and what is not enforced

The word "budget" has three meanings here. Confusing them can lead to false confidence about cloud spending.

Budget type	Current implementation
Per-call generation budget	Enforced through provider-native output ceilings
Retry/attempt budget	One initial call + max_repairs; repairs are clamped to 0-5
Money / daily account budget	Not enforced by this app; no pricing configuration, hard spend cap or account-wide quota monitor

Simple bounded-output arithmetic

```
maximum attempts = 1 + max_repairs
nominal output allowance = per-call ceiling * maximum attempts

Example with max_repairs = 5:
Single request: 2,000 * 6 = 12,000
Data or Load:  4,000 * 6 = 24,000
Suite:          6,000 * 6 = 36,000
```

These numbers are nominal output allowances across attempts, **not total billable tokens or prices**. Every call also processes input; repeated context is billed according to the provider's pricing/cache rules. Truncation now stops the loop immediately, so this worst-case multiplication does not apply to six identical truncated calls.

What restrains consumption today?

- Local retrieval sends selected cards rather than the complete catalog.
- Per-workflow output ceilings bound each generation request.
- Repair rounds bound ordinary retries.
- Sarvam structured plans disable reasoning.
- Hosted truncation stops without automatic budget escalation.
- Cloud consent and explicit Ask govern calls.
- Usage history exposes actual reported consumption, including hosted checks and failures with reported counts.

Illustrative cost formula - not implemented pricing

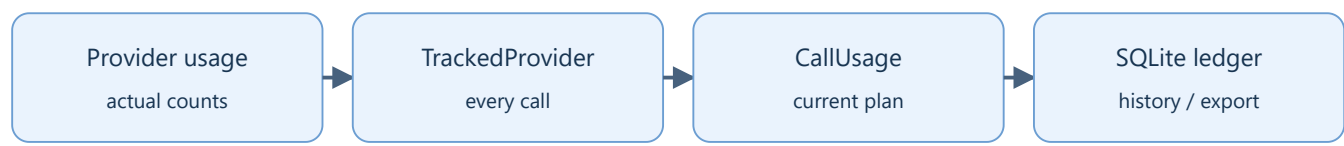
```
estimated cost =
  (billable input tokens / 1,000,000) * input rate
+ (billable output tokens / 1,000,000) * output rate
+ provider-specific cache / service adjustments
```

You must obtain current model-specific rates from the provider. The dashboard deliberately does not apply a generic price or label unknown cost as zero. Ollama has no cloud API charge in the local configuration, but hardware, electricity and infrastructure costs still exist.

Operational recommendation: set provider-side quota/spend alerts, use a modest repair setting, and review usage after long suites. These are recommended controls, not features already implemented. Timeout is per call; there is no end-to-end generation deadline or daily token quota in the current code.

18. Usage accounting and history

Model-call metadata flows to two separate views



There are two views of usage. Token details describes one generation, including its repair calls. AI Usage is the persistent ledger of calls made by this application across generations and connection checks.

What the provider reports

Provider	Usage fields read
OpenAI / Azure / Sarvam	prompt_tokens and completion_tokens
Claude	input_tokens plus reported cache_creation_input_tokens/cache_read_input_tokens; output_tokens
Ollama	prompt_eval_count and eval_count

The common `LlmResponse` carries normalized counts and flags saying whether counts were actually reported. Missing values display as Not reported, not fabricated zero. Reasoning-token breakdown and per-category pricing are not displayed separately in the current common usage contract.

Current-plan detail

`CallUsage` stores purpose, input/output counts (nullable), duration, model and status. Status distinguishes Completed, Invalid response and Truncated response. The dialog totals only this generation and shows each call. If a needed count is unknown, a complete numeric total is not invented. Time is accumulated model-call wall time, not necessarily the complete UI operation duration.

Persistent ledger

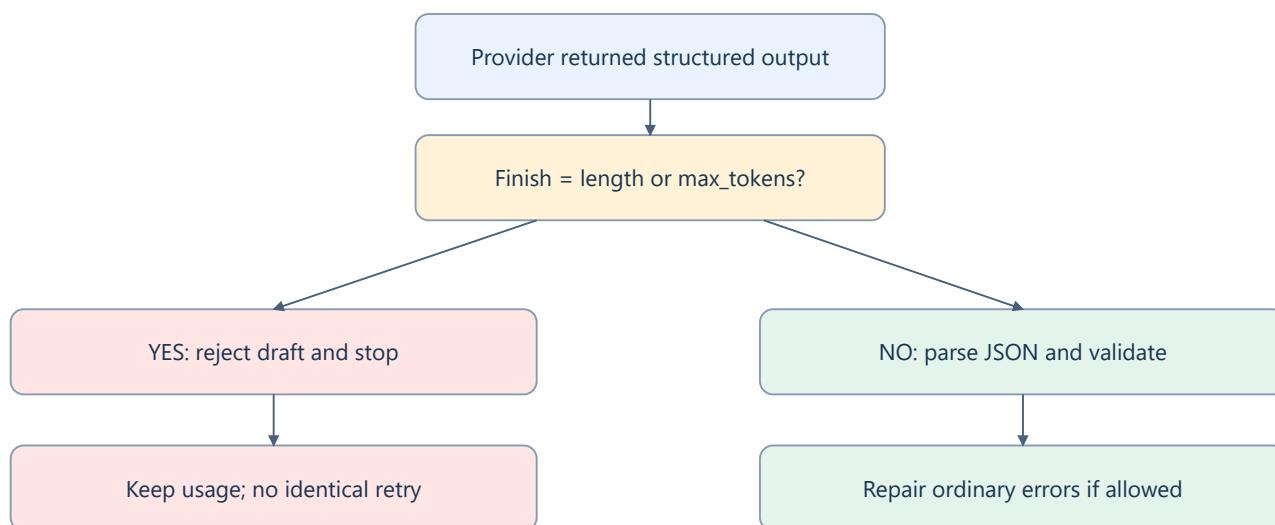
`TrackedProvider.complete` wraps every call. SQLite records timestamps, session/generation IDs, connection/provider/model identity, workflow kind, purpose, counts, duration, status and ready flag. Failed hosted responses with diagnostics preserve counts; failures without usage remain unknown. A generation is marked ready only for a validated status=plan outcome, not for a clarification, unsupported result or executed API run.

History filters include Today, 7/30 days, This session and All recorded. Export writes filtered metadata to CSV, neutralizing dangerous spreadsheet formula prefixes in user-controlled text. Records survive connection deletion. There is no pre-upgrade history reconstruction or provider-account synchronization.

Analogy: a trip receipt and a monthly expense ledger. Token details is the receipt for this trip. AI Usage is the ledger of trips this app recorded. Neither sees trips made in another application.

Stored ledger metadata excludes prompts, keys, endpoint URLs and raw response content. User-chosen connection names are still metadata that may reveal organizational context; treat exports appropriately.

19. Truncation: diagnosis and correction



The observed Sarvam failure

A Load plan used a 4,000-token ceiling and normal default reasoning. Two calls took about 35-37 seconds each, returned a token-limit finish reason, and did not produce valid plan JSON. The old loop classified this as ordinary invalid JSON and requested another repair at the same ceiling. The log established truncation, not the exact reasoning-versus-final-text split.

What changed

1. **Prevention:** Sarvam schema-based plan calls now explicitly send `reasoning_effort: null`. The 2,000/4,000/6,000 ceilings were not raised.
2. **Detection:** hosted finish reasons `length` and `max_tokens` are classified as truncation before accepting structured output.
3. **Metadata:** `LlmResponseError` carries a diagnostics-only response: known token counts, reported flags, duration, whitelisted finish reason and empty/non-empty final text.
4. **Stop:** request/workflow and suite loops record `output_truncated` and break immediately.
5. **UI:** show an actionable smaller-plan/model message, retain failed usage and block handoff.

```
# Implementation logic, simplified:
if response_schema is not None and finish_reason in {"length", "max_tokens"}:
    raise LlmResponseError("Output token limit reached.",
                           diagnostics=metadata_only_response)

except LlmResponseError as exc:
    record_failure_and_reported_usage(exc)
    if exc.truncated:
        break # Do not spend another call on the same budget.
```

Even parseable JSON is rejected when the provider reports truncation: syntactic closure does not prove the intended plan is complete. No partial-plan execution, fabricated JSON completion or silent success fallback occurs.

What remains: a genuinely large final plan can still exceed its cap. The user must simplify the goal, split it into smaller generations or select another compatible model. Increasing timeout will not fix a token-limit finish. Normal non-truncated JSON/catalog errors still use bounded repair rounds.

20. Activity, errors and notifications

Activity is an application event log

It records relative timestamps for retrieval, call completion/failure, endpoint selection, validation locations/codes, refinement reasons and final outcomes. It is not the model's private reasoning. Entries are sanitized, capped to 300 text blocks, and messages are bounded. Copy activity exports the visible log.

```
[ 0.0s] Starting load test planning; Sarvam AI / sarvam-105b
[ 0.0s] Retrieved 8 endpoint candidates; showing 3 detailed cards
[ 12.4s] Call 1 returned ... input tokens: 900; output tokens: 430
[ 12.4s] Validating against the catalog
[ 12.4s] Catalog validation passed
[ 12.4s] Generation ended ... Nothing executed.
```

This timeline is illustrative, not a recorded benchmark. Relative elapsed time is not a provider-server timing breakdown. Full messages are in the drawer; the compact progress label shortens lengthy text.

Safe failure visibility

Validation Activity logs issue paths and codes rather than dumping offending payload values. Known secrets and credential-like patterns are masked. Hosted error bodies are not printed. Unexpected worker exceptions are surfaced as failures; users should still treat copied diagnostic metadata as internal information.

Notifications and snapshots

When planning completes, the main window stops the header animation and stores a bounded in-memory snapshot. The notification bell can reopen the exact result, including the HTML preview, outcome, token calls, Activity, connection identity and relevant CSV header settings. Restoring does not contact the provider or execute an API.

The current result history is session memory, not a persistent conversation transcript. The SQLite ledger is persistent, but it cannot reconstruct the full plan because it deliberately does not store prompts or response bodies.

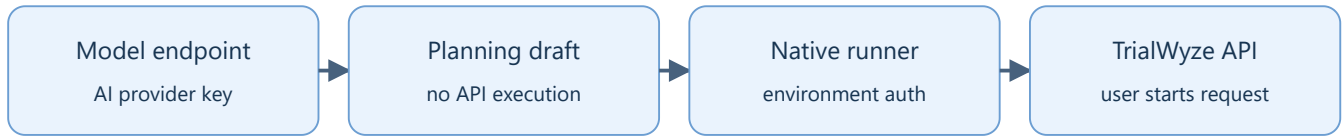
Analogy: a parcel tracking page. You see "received", "sorted", "sent" and "delivered", not the courier's private thoughts. The status explains progress without exposing private contents.

Success needs interpretation

A worker job can return a valid unsupported/clarification outcome without throwing an exception. A ready plan requires both no outstanding issues and status=plan. "Generation finished" is not the same as "API test passed". The native runner's result model separately distinguishes pass, failure, error and skipped cases.

21. Safety and privacy boundaries

Two networks, two different credentials



The AI provider endpoint and the TrialWyze API endpoint are different networks. Their credentials must not be confused. The model adapter authenticates the planning call. Native API execution uses the user's environment configuration, access token, API key and other existing authentication controls.

Implemented defenses

- Hosted AI requires explicit consent before sending context.
- Provider keys use encrypted persistence or environment bindings, not normal settings JSON.
- Outbound messages are redacted with known secrets and credential-pattern matching.
- CSV workflow sends headings only and rejects sensitive/file-path mappings.
- Model output is data; the app does not eval it as Python or run arbitrary tool code.
- Catalog and workflow validators check what the app can represent.
- Native handoff requires review; Data/Load revalidate current contracts and preserve execution guards.
- Load permission/write/approval checkboxes are cleared on import.
- Diagnostics store metadata, not raw provider bodies.

What redaction actually does

Known explicit secrets of at least four characters are replaced, longest first. Regex patterns cover common JWT, Bearer/Basic, password/key assignments and long key-like sequences. `redact_messages` runs before provider calls. The controller includes the actual provider key in the secrets given to the planner.

Redaction is best effort, not a data-loss-prevention guarantee. Unrecognized personal data, short secrets, sensitive prose, resource names or schema descriptions may not be masked. Do not paste production data or confidential records into prompts. Cloud consent does not mean all information is automatically safe.

The limits of a catalog validator

A catalog is a trusted snapshot, not a live security oracle. It cannot prove permission, business-policy safety, tenant isolation, record existence or safe server capacity. Prompt instructions are not a sandbox. Missing schema coverage and heuristic response fields require human review. Load numeric maximums are parsing bounds, not safe production recommendations.

Generated write tests should run only in authorized environments. Cleanup can fail; maintain a traceable test-data prefix and a manual cleanup plan. Audit provider-side retention policies independently; the local app's metadata-only ledger does not define a cloud provider's retention behavior.

22. Issues faced and how they were fixed

Issue / symptom	Cause	Correction
Sarvam connection shows empty answer; icon stays dim	32-token probe left too little room for default reasoning/final text	Disable Sarvam probe reasoning, 128-token cap, 30s minimum; activate icon only on success
Other reasoning-provider probes fail	Tiny output cap and forced sampling settings are incompatible with some models	4,096-token / 60s probes; omit temperature and do not force universal thinking settings
Load plan repeats truncation	Truncation treated as ordinary invalid JSON at unchanged cap	Disable Sarvam structured reasoning; typed finish diagnostics; stop first truncation
Failed usage looks unknown despite provider counts	JSON error discarded response usage	Metadata-only diagnostics retained in per-plan details and tracked ledger
Suite is much slower than one request	More endpoint cards/cases, larger output, possible sequential repair	Observable Activity and bounded retries; not a claim that all suite latency is eliminated
Small local context loses suite facts	Default server window too small for detailed schema context	Explicit configurable Ollama num_ctx, default 16,384; compact retrieval
Load draft cannot round-trip null SLA override	Native stage serialization includes nullable field	Accept null representation; review native inclusion behavior rather than inventing override semantics
CSV constants overwrite mapped IDs	Missing path handling did not account for mapped sources	Mapping-aware path handling; headers rechecked before handoff
Token details seems unresponsive before result	No usage strip populated yet	Always-clickable placeholder explains unavailable data; real result opens detail table
Warning uses only a narrow center strip	Equal layout stretches deprived label of width	Expanding warning group centered with bulb attached and available width
Provider wraps around settings gear	Word-wrap label compressed in control row	Reusable single-line ElidingLabel, full tooltip and full-text metadata
Window freezes during planning	Slow network/model work would block UI thread	QThread worker and Qt signals; cooperative cancellation

Evidence boundary: fixes were verified with fake providers/mock transport and focused Qt checks. The original Sarvam log confirms token-limit termination but not an exact internal reasoning count. No live provider benchmark is claimed. Three unrelated older regressions remained during earlier checks: two obsolete mode-button UI tests and one SLA-stage expectation. They were not repaired as part of truncation handling.

Historical UI changes: the earlier large example cards were removed during the minimal composer redesign. The Close action was removed; native window close remains. The yellow bulb is reusable in several colors, and disabled Delete actions keep the standard red icon. These are UX changes, not changes to the planning algorithm.

23. Code examples and extension guide

Small offline example using the existing fake provider

```
import json
from pathlib import Path
from api_tester.catalog import load_catalog
from api_tester.ai.catalog_index import CatalogIndex
from api_tester.ai.config import AiSettings
from api_tester.ai.planner import RequestPlanner
from api_tester.ai.provider import LlmResponse
from api_tester.ai.providers.fake import FakeProvider
from api_tester.ai.request_plan import RequestPlan

# Run from the repository root; this example uses the bundled store demo.
catalog = load_catalog(Path("examples") / "store_catalog.json")
value = RequestPlan(endpoint_id="f76aeb49ca37").to_json()
provider = FakeProvider([LlmResponse(
    text=json.dumps(value), json=value,
    input_tokens=20, output_tokens=10, model="fake"
)])
outcome = RequestPlanner(
    provider, CatalogIndex(catalog), AiSettings(max_repairs=0),
    progress=print
).plan("list products")
assert outcome.ok
print(outcome.plan.to_json())
# No provider network call. No TrialWyze API execution.
```

Key methods to explain to a developer

Method	Input -> output
CatalogIndex.search	Goal -> ranked SearchHit objects
CatalogIndex.card	Endpoint -> compact factual text
RequestPlanner.plan	Prompt -> PlanOutcome with draft/issues/usage
WorkflowPlanner.parse_plan / validate_plan	JSON -> WorkflowPlan -> checked result
LlmProvider.complete	Messages + schema + caps -> LlmResponse or safe error
TrackedProvider.complete	Same call -> response plus ledger entry
MainWindow.open_ai_load / open_ai_data	Checked plan -> native editable draft, never execution

Adding another provider: implement the common interface, map auth/schema/usage/finish reasons, register profile defaults and hints, and add mocked wire/consent/failure tests. Do not hard-code keys or assume all OpenAI-compatible services support the same reasoning settings.

Adding a workflow: define a representable typed plan, strict parser/validator, prompt/schema, safe native converter, human-controlled handoff, usage kind, UI mode and offline regressions. Reuse the planner hooks instead of duplicating retrieval, redaction, cancellation and truncation behavior.

24. Testing and a novice demonstration

What was verified for the recent changes

Focused offline checks exercised Sarvam/auth wire fields, probe budgets/timeouts, default-reasoning compatibility, malformed Ollama lists, icon availability, normal repair behavior, truncation stops, reported/unknown usage, safe diagnostics, native handoff blocking, snapshots and single-line composer layout. The truncation-focused combined selection passed 123 tests after explicitly excluding three unrelated existing failures; composer layout added two focused passing checks. These are historical task results, not a newly certified full application suite.

Test file	Purpose
test_request_planner.py / test_suite_planner.py	Catalog plans, validation, refinements and suite behavior
test_workflow_planner.py / test_workflow_handoff.py	Data/Load parsing, supported models and native draft transfer
test_sarvam_provider.py / test_connection_probes.py	Provider contract, credentials/consent, probes and toolbar status
test_activity.py / test_truncation.py	Safe events, finish reasons, one-call stop and preserved failure usage
test_composer_layout.py	Warning geometry, gear callback/busy guard, provider single-line display

A safe ten-minute demonstration

1. Open a catalog and show one endpoint's actual filters/payload fields.
2. Show AI Settings: active profile, cloud consent, Test connection and its billing note.
3. Ask for one read-only request with a known test ID.
4. Explain Activity versus Token details. Point out assumptions.
5. Open the draft in API Explorer; emphasize that no API call has happened.
6. Run only against an approved test environment; show native results.
7. Show a small suite capture feeding a later case.
8. Choose a sample CSV; explain headings-only AI input and local row validation.
9. Show a tiny load draft and its cleared approvals. Do not start a load demo without permission.
10. Open AI Usage and explain per-app accounting, partial counts and absence of monetary pricing.

How to answer common questions

"Is AI reading our database?" Not through this planner; it sees selected catalog facts and your prompt. **"Can it run tests by itself?"** No: it produces drafts and native execution requires user action. **"Does valid JSON mean a good test?"** No: local checks and human review are still required. **"Why does Cancel wait?"** In-flight non-streaming calls are cooperative, not immediately aborted. **"Are we protected by a daily money cap?"** Not by this app; configure the provider's account controls.

25. Glossary, references and limitations

Term	Easy definition
LLM / provider	A text-generating model / the service hosting it
Catalog grounding	Giving AI selected facts from known API contracts
Schema	A description of permitted data shape/fields
DTO / dataclass	A structured container for data
Validation	Deterministic checks on syntax, values and supported behavior
Repair	A bounded request to correct a previous invalid proposal
Refinement	Another call with additional endpoint facts or missing-value guidance
Truncation	Generation stopped at a token ceiling before completion
Capture / dependency	Save a response value / require an earlier case
Virtual user / think time	Simulated client / pause between that client's requests
p95 / error rate	95th-percentile latency / fraction of errors
Ledger / snapshot	Persistent metadata history / session-only result copy

Official provider references

- [OpenAI reasoning and token accounting](#)
- [OpenAI model/parameter compatibility](#)
- [Azure OpenAI reasoning models](#)
- [Claude Messages API](#)
- [Claude thinking and model differences](#)
- [Sarvam V1 chat: reasoning, limits and formats](#)
- [Ollama model-list endpoint](#)
- [Ollama native chat API](#)

Documentation changes over time. The verified local adapter behavior takes precedence over assumptions about a future model. The guide describes this working-tree snapshot, not universal compatibility.

Current limitations, not hidden promises

- No exact local model-token counting, automatic context-fit calculation or intelligent transcript pruning.
- No daily monetary/token cap, current pricing engine or provider-account quota synchronization.
- No private reasoning viewer, streamed partial-plan UI or immediate HTTP cancellation.
- No automatic model switching, truncation continuation or dynamic output-budget escalation.
- No live database discovery, automatic database seeding, service-code fine-tuning or independent API execution.
- Catalog coverage, heuristic response fields, native representability and model compatibility can limit plans.

Final teaching sentence: "AI drafts a test using our API cookbook; our app checks the draft; we review it; only the normal testing tools execute it."

Appendix A. Source-derived function and method index

This index is generated from Python syntax trees, not invented by the model. Line numbers identify this snapshot. A leading underscore means an internal helper. The short role uses the function docstring when available; otherwise the owning module provides its context. These are identifiers, not a recommendation to call private APIs.

api_tester/ai/catalog_index.py

Function / method signature	Line	Role / reference
<code>split_words(text: str)</code>	42	Internal formatting/check helper; see owning module in Chapter 3.
<code>singular(word: str)</code>	50	Internal formatting/check helper; see owning module in Chapter 3.
<code>tokens(text: str)</code>	60	Normalize local search words; not model tokenizer IDs.
<code>query_tokens(text: str)</code>	64	Remove search stopwords and very short words.
<code>SearchHit.summary(self)</code>	73	Internal formatting/check helper; see owning module in Chapter 3.
<code>endpoint_summary(endpoint: Endpoint)</code>	77	Create a compact one-line candidate description.
<code>CatalogIndex.__init__(self, catalog: Catalog)</code>	85	Initialize this component's state and dependencies.
<code>CatalogIndex.get(self, endpoint_id: str)</code>	105	Look up an endpoint by its catalog identity.
<code>CatalogIndex._near_words(self, word: str)</code>	108	Find fuzzy search neighbors with similar spelling.
<code>CatalogIndex._document(self, endpoint: Endpoint)</code>	117	Assign relevance weights to catalog words.
<code>CatalogIndex.search(self, query: str, *, limit: int=8, service: str None=None, method: str None=None)</code>	136	Rank catalog endpoints locally using words, weights and method hints.
<code>CatalogIndex.filter_schema(self, endpoint: Endpoint)</code>	181	Internal formatting/check helper; see owning module in Chapter 3.
<code>CatalogIndex.card(self, endpoint: Endpoint, *, max_filter_fields: int=40, max_lov: int=12, include_response: bool=False)</code>	188	Describe supported parameters, fields and schemas for one endpoint.
<code>CatalogIndex.response_text(self, endpoint: Endpoint, max_lov: int=12)</code>	235	Describe response coverage or clearly marked likely fields.
<code>CatalogIndex.likely_record_fields(self, endpoint: Endpoint, limit: int=30)</code>	248	Best guess at a resource's record fields when no response schema is declared.
<code>CatalogIndex.siblings(self, endpoint: Endpoint)</code>	260	Every endpoint on the same service controller - one resource's full lifecycle.
<code>is_generic_schema(schema: dict[str, Any])</code>	269	Internal formatting/check helper; see owning module in Chapter 3.
<code>_parameter_text(parameter, max_lov: int)</code>	273	Internal formatting/check helper; see owning module in Chapter 3.
<code>_filter_field_text(field, max_lov: int)</code>	280	Internal formatting/check helper; see owning module in Chapter 3.
<code>_payload_field_names(schema: dict[str, Any] None)</code>	288	Internal formatting/check helper; see owning module in Chapter 3.

Function / method signature	Line	Role / reference
<code>_payload_text(schema: dict[str, Any], max_lov: int, depth: int=0)</code>	296	Internal formatting/check helper; see owning module in Chapter 3.
<code>_truncate(text: str, limit: int)</code>	317	Internal formatting/check helper; see owning module in Chapter 3.

api_tester/ai/config.py

Function / method signature	Line	Role / reference
<code>AiConnection.create(cls, name: str, provider: str='ollama')</code>	52	Create a named provider connection with registered defaults.
<code>AiConnection.from_dict(cls, value: dict[str, Any])</code>	60	Load/validate configuration data into a profile.
<code>AiSettings.from_dict(cls, value: dict[str, Any] None)</code>	100	Load/validate configuration data into a profile.
<code>AiSettings.to_dict(self)</code>	137	Serialize the active connection/configuration metadata.
<code>AiSettings.ensure_connections(self)</code>	141	Migrate the single-provider format without changing its active choice or key.
<code>AiSettings.for_connection(self, connection_id: str)</code>	164	Apply the chosen named profile to shared planning settings.
<code>AiSettings.for_provider(self, provider: str)</code>	173	Resolve a provider profile during configuration/migration.
<code>load_ai_settings(path: Path, *, strict: bool=False)</code>	184	Internal formatting/check helper; see owning module in Chapter 3.
<code>save_ai_settings(path: Path, settings: AiSettings)</code>	200	Internal formatting/check helper; see owning module in Chapter 3.
<code>build_provider(settings: AiSettings, *, api_key: str='')</code>	213	Resolve consent/credentials and construct a matching adapter.
<code>_credential_path(credential_id: str)</code>	228	Internal formatting/check helper; see owning module in Chapter 3.
<code>get_ai_key(provider: str, credential_id: str='', key_environment: str='')</code>	234	Read bound environment key or encrypted credential; surface failure.
<code>save_ai_key(provider: str, value: str, credential_id: str='')</code>	253	Persist a credential through OS-encrypted storage.
<code>delete_ai_key(credential_id: str)</code>	262	Remove the selected stored credential file.

api_tester/ai/controller.py

Function / method signature	Line	Role / reference
<code>_Worker.__init__(self, job: Callable[['_Worker'], Any])</code>	29	Initialize this component's state and dependencies.
<code>_Worker.run(self)</code>	35	Execute worker job and emit result/error while respecting cancellation.
<code>AiTask.__init__(self, job: Callable[['_Worker'], Any], parent: QObject None=None)</code>	59	Initialize this component's state and dependencies.
<code>AiTask.start(self)</code>	72	Start the task's worker thread.
<code>AiTask.cancel(self)</code>	75	Set cooperative stop flag; do not claim an immediate HTTP abort.
<code>AiTask.is_running(self)</code>	78	Report worker-thread running state.
<code>AiTask.wait(self, milliseconds: int=0)</code>	81	Wait for the worker thread to finish.

Function / method signature	Line	Role / reference
<code>plan_request_task(prompt: str, settings: AiSettings, index: CatalogIndex, secrets: tuple[str, ...], parent: QObject None=None)</code>	85	Internal formatting/check helper; see owning module in Chapter 3.
<code>plan_suite_task(prompt: str, settings: AiSettings, index: CatalogIndex, secrets: tuple[str, ...], parent: QObject None=None)</code>	111	Internal formatting/check helper; see owning module in Chapter 3.
<code>plan_workflow_task(prompt: str, settings: AiSettings, index: CatalogIndex, secrets: tuple[str, ...], *, mode: str, columns: tuple[s...])</code>	136	Internal formatting/check helper; see owning module in Chapter 3.
<code>connection_test_task(settings: AiSettings, parent: QObject None=None, *, api_key: str='')</code>	165	Internal formatting/check helper; see owning module in Chapter 3.

api_tester/ai/planner.py

Function / method signature	Line	Role / reference
<code>PlanOutcome.ok(self)</code>	60	Report whether this result has outstanding validation issues.
<code>RequestPlanner.__init__(self, provider: LlmProvider, index: CatalogIndex, settings: AiSettings, *, secrets: Iterable[str]=(), progress: ...)</code>	65	Initialize this component's state and dependencies.
<code>RequestPlanner._check_cancelled(self)</code>	82	Stop at a cooperative planning cancellation checkpoint.
<code>RequestPlanner._record_usage(self, outcome, attempt: int, response, elapsed_ms: int, status: str='Completed')</code>	86	Record one plan call and emit safe consumption diagnostics.
<code>RequestPlanner._record_response_failure(self, outcome, attempt: int, started: float, exc: LlmResponseError)</code>	107	Keep failure usage, classify truncation and stop identical retries.
<code>RequestPlanner._validation_activity(self, issues)</code>	132	Emit issue locations/codes without dumping invalid values.
<code>RequestPlanner._response_failure(self, attempt: int, started: float, exc: Exception)</code>	141	Emit a sanitized call-failure event with elapsed time.
<code>RequestPlanner.system_prompt(self)</code>	147	Choose instructions for the workflow.
<code>RequestPlanner.response_schema(self, candidate_ids: list[str])</code>	150	Build the model's expected JSON contract.
<code>RequestPlanner.parse_plan(self, value: Any)</code>	153	Parse the selected workflow's model output.
<code>RequestPlanner.validate_plan(self, plan: RequestPlan)</code>	156	Run deterministic checks against catalog/workflow rules.
<code>RequestPlanner.output_token_limit(self)</code>	159	Return the fixed per-call workflow generation ceiling.
<code>RequestPlanner.plan(self, prompt: str)</code>	162	Coordinate a planning operation and return outcome, issues and usage.
<code>_missing_path_parameters(plan: RequestPlan, endpoint: Endpoint None)</code>	265	Internal formatting/check helper; see owning module in Chapter 3.
<code>SuiteOutcome.ok(self)</code>	287	Report whether this result has outstanding validation issues.
<code>SuitePlanner.candidates(self, prompt: str)</code>	300	(offered endpoints, endpoints shown as full cards).
<code>SuitePlanner._card(self, endpoint: Endpoint)</code>	323	Internal formatting/check helper; see owning module in Chapter 3.
<code>SuitePlanner.plan(self, prompt: str)</code>	326	Coordinate a planning operation and return outcome, issues and usage.

api_tester/ai/prompts.py

Function / method signature	Line	Role / reference
<code>request_user_message(prompt: str, summaries: list[str], cards: list[str])</code>	60	Internal formatting/check helper; see owning module in Chapter 3.
<code>suite_user_message(prompt: str, summaries: list[str], cards: list[str])</code>	127	Internal formatting/check helper; see owning module in Chapter 3.

api_tester/ai/provider.py

Function / method signature	Line	Role / reference
<code>LlmResponseError.__init__(self, message: str, *, diagnostics: LlmResponse None=None)</code>	65	Initialize this component's state and dependencies.
<code>LlmResponseError.truncated(self)</code>	70	Recognize length/max_tokens from metadata-only error diagnostics.
<code>LlmProvider.complete(self, messages: list[ChatMessage], *, model: str, tools: list[ToolSpec] None=None, response_schema: dict[str, ...])</code>	86	Execute or wrap one provider model call using the common contract.
<code>LlmProvider.check_connection(self, model: str, timeout_seconds: float=10.0)</code>	98	Check provider/model availability and return an explicit report.
<code>probe_hosted_connection(provider: LlmProvider, model: str, timeout_seconds: float=15.0)</code>	101	Run one bounded final-text probe with provider-aware limits.

api_tester/ai/redaction.py

Function / method signature	Line	Role / reference
<code>redact_text(text: str, secrets: Iterable[str]=())</code>	29	Mask known secrets and credential-shaped text.
<code>_mask_match(match: re.Match[str])</code>	40	Internal formatting/check helper; see owning module in Chapter 3.
<code>redact_messages(messages: list[ChatMessage], secrets: Iterable[str]=())</code>	48	Apply masking to every outbound message's text.

api_tester/ai/request_plan.py

Function / method signature	Line	Role / reference
<code>RequestPlan.from_json(cls, value: Any)</code>	54	Parse model JSON into the component's typed plan.
<code>RequestPlan.to_json(self)</code>	91	Serialize a typed draft into its machine-readable plan representation.
<code>RequestPlan.with_changes(self, **changes: Any)</code>	102	Return a modified copy rather than mutate the immutable draft.
<code>_text(value: Any)</code>	106	Internal formatting/check helper; see owning module in Chapter 3.
<code>_list(value: Any)</code>	114	Internal formatting/check helper; see owning module in Chapter 3.
<code>_payload(value: Any)</code>	118	Internal formatting/check helper; see owning module in Chapter 3.
<code>request_plan_schema(candidate_ids: list[str])</code>	133	JSON Schema enforced by the provider.
<code>to_explorer_request(plan: RequestPlan, endpoint: Endpoint, structured: bool)</code>	195	Converts a validated plan into the API Explorer draft for "endpoint".

api_tester/ai/suite_plan.py

Function / method signature	Line	Role / reference
SuiteCasePlan.request_plan(self)	75	Internal formatting/check helper; see owning module in Chapter 3.
SuiteCasePlan.from_json(cls, value: dict[str, Any], position: int)	88	Parse model JSON into the component's typed plan.
SuiteCasePlan.to_json(self)	122	Serialize a typed draft into its machine-readable plan representation.
SuitePlan.from_json(cls, value: Any)	152	Parse model JSON into the component's typed plan.
SuitePlan.to_json(self)	182	Serialize a typed draft into its machine-readable plan representation.
SuitePlan.with_changes(self, **changes: Any)	196	Return a modified copy rather than mutate the immutable draft.
suite_plan_schema(candidate_ids: list[str])	200	Internal formatting/check helper; see owning module in Chapter 3.
SuiteValidation.ok(self)	275	Report whether this result has outstanding validation issues.
_variables_in(value: Any)	279	Internal formatting/check helper; see owning module in Chapter 3.
_case_variables(case: SuiteCasePlan)	289	Internal formatting/check helper; see owning module in Chapter 3.
_is_success(expected: str)	296	Internal formatting/check helper; see owning module in Chapter 3.
response_path_problem(schema: dict[str, Any] None, path: str)	300	Why ``path`` cannot exist in ``schema``; ``None`` when it can (or the shape is unknown).
validate_suite_plan(plan: SuitePlan, index: CatalogIndex)	336	Check cases, assertions, captures and dependencies.
_ancestors(key: str, graph: dict[str, list[str]])	540	Internal formatting/check helper; see owning module in Chapter 3.
_guess_variable(controller: str)	552	Internal formatting/check helper; see owning module in Chapter 3.
to_test_suite(plan: SuitePlan, index: CatalogIndex)	560	Converts a validated plan into a runnable :class:`TestSuite`.

api_tester/ai/ui.py

Function / method signature	Line	Role / reference
_keep_alive(task: AiTask)	84	Internal formatting/check helper; see owning module in Chapter 3.
shutdown_ai_tasks(milliseconds: int=2000)	90	Internal formatting/check helper; see owning module in Chapter 3.
AskAiDialog.__init__(self, parent: QWidget None, *, catalog: Callable[[], Catalog], secrets: Callable[[], tuple[str, ...]], on_open...)	100	Initialize this component's state and dependencies.
AskAiDialog._build(self)	147	Create this component's Qt widgets and signal wiring.
AskAiDialog.refresh_theme(self)	320	Refresh themed colors/icons and layout presentation.
AskAiDialog.refresh_connection_state(self)	355	Internal formatting/check helper; see owning module in Chapter 3.
AskAiDialog.set_mode(self, mode: str)	366	Select supported workflow and clear stale outcomes.
AskAiDialog._choose_csv(self)	403	Read/validate selected CSV headers without transmitting row data.
AskAiDialog.current_settings(self)	428	Internal formatting/check helper; see owning module in Chapter 3.

Function / method signature	Line	Role / reference
<code>AskAiDialog.reload_settings(self)</code>	431	Refresh the active provider's saved configuration.
<code>AskAiDialog._manage_settings(self)</code>	441	Open AI Settings when permitted and reload active profile.
<code>AskAiDialog._set_connection(self, message: str, color: str)</code>	446	Internal formatting/check helper; see owning module in Chapter 3.
<code>AskAiDialog.index(self)</code>	452	Internal formatting/check helper; see owning module in Chapter 3.
<code>AskAiDialog.ask(self)</code>	459	Collect goal, start correct task and clear stale generation state.
<code>AskAiDialog._cancel(self)</code>	521	Request cooperative cancellation and log the user action.
<code>AskAiDialog._on_progress(self, text: str)</code>	527	Append Activity and update compact progress/timer.
<code>AskAiDialog._tick(self)</code>	534	Internal formatting/check helper; see owning module in Chapter 3.
<code>AskAiDialog._on_failed(self, message: str)</code>	539	Show escaped explicit generation failure.
<code>AskAiDialog._on_done(self)</code>	546	Finish busy state and capture notification/history snapshot.
<code>AskAiDialog.restore_result(self, snapshot: dict)</code>	578	Restore a completed generation without sending another model call.
<code>AskAiDialog._append_activity(self, message: str)</code>	602	Sanitize and timestamp one bounded diagnostic entry.
<code>AskAiDialog._set_busy(self, busy: bool)</code>	607	Protect editing controls during generation.
<code>AskAiDialog._show_idle(self)</code>	621	Internal formatting/check helper; see owning module in Chapter 3.
<code>AskAiDialog._refresh_buttons(self)</code>	633	Enable copying/handoff only for applicable current outcomes.
<code>AskAiDialog._show_token_details(self)</code>	642	Internal formatting/check helper; see owning module in Chapter 3.
<code>AskAiDialog._on_workflow_outcome(self, outcome: PlanOutcome)</code>	657	Render Data/Load result and prepare valid native handoff.
<code>AskAiDialog._on_outcome(self, outcome: PlanOutcome)</code>	670	Render single request proposal and native editor conversion.
<code>AskAiDialog._on_suite_outcome(self, outcome: SuiteOutcome)</code>	684	Render suite and prepare native suite conversion.
<code>AskAiDialog._show_meta(self, outcome)</code>	699	Show provider identity and generation usage.
<code>AskAiDialog._offer_choices(self, status: str, choices)</code>	707	Render clarification options when the plan asks a question.
<code>AskAiDialog._answer(self, choice: str)</code>	716	Append clarification answer to the goal and regenerate explicitly.
<code>AskAiDialog._clear_choices(self)</code>	723	Internal formatting/check helper; see owning module in Chapter 3.
<code>AskAiDialog._copy_plan(self)</code>	730	Copy machine-readable current draft.

Function / method signature	Line	Role / reference
AskAiDialog._open(self)	740	Hand off selected draft; this is not an execution action.
AskAiDialog.closeEvent(self, event)	760	Internal formatting/check helper; see owning module in Chapter 3.
_list(items, color: str None=None)	766	Internal formatting/check helper; see owning module in Chapter 3.
render_outcome(outcome: PlanOutcome, endpoint, request: ExplorerRequest None)	771	Internal formatting/check helper; see owning module in Chapter 3.
render_suite_outcome(outcome: SuiteOutcome, index: CatalogIndex, conversion_error: str='')	831	Internal formatting/check helper; see owning module in Chapter 3.

api_tester/ai/usage.py

Function / method signature	Line	Role / reference
CallUsage.total_tokens(self)	16	Add input/output only when both counts are reported.
token_text(value: int None)	22	Internal formatting/check helper; see owning module in Chapter 3.
usage_total(calls: list[CallUsage], field: str)	26	Internal formatting/check helper; see owning module in Chapter 3.

api_tester/ai/usage_dashboard.py

Function / method signature	Line	Role / reference
totals_text(rows: list[dict], field: str)	18	Display known history totals while disclosing unknown counts.
UsageDashboard.__init__(self, parent=None, *, store: UsageStore None=None)	27	Initialize this component's state and dependencies.
UsageDashboard.refresh(self, _index=None)	81	Reload/filter/group ledger metadata for the usage dashboard.
UsageDashboard.export(self)	123	Export filtered ledger rows, guarding spreadsheet formula prefixes.

api_tester/ai/usage_store.py

Function / method signature	Line	Role / reference
UsageStore.__init__(self, path: Path=USAGE_PATH)	21	Initialize this component's state and dependencies.
UsageStore._connect(self)	34	Internal formatting/check helper; see owning module in Chapter 3.
UsageStore.record(self, settings: AiSettings, generation_id: str, kind: str, purpose: str, response: LlmResponse None, duration_...)	39	Insert a metadata-only per-call usage row.
UsageStore.mark_plan_ready(self, generation_id: str)	58	Mark generation as validated plan, not executed API test.
UsageStore.rows(self, *, since: datetime None=None, session_id: str='')	62	Read usage history with optional time/session filter.
TrackedProvider.__init__(self, provider: LlmProvider, settings: AiSettings, kind: str, store: UsageStore None=None)	80	Initialize this component's state and dependencies.
TrackedProvider.complete(self, messages: list[ChatMessage], *, model: str, tools: list[ToolSpec] None=None, response_schema: dict[str, ...])	90	Execute or wrap one provider model call using the common contract.

Function / method signature	Line	Role / reference
TrackedProvider.check_connection(self, model: str, timeout_seconds: float=10.0)	113	Check provider/model availability and return an explicit report.

api_tester/ai/usage_ui.py

Function / method signature	Line	Role / reference
UsageStrip.__init__(self, parent=None, *, compact: bool=False)	18	Initialize this component's state and dependencies.
UsageStrip.show_usage(self, calls: list[CallUsage], connection: str, provider: str)	53	Populate this generation's counts and call detail.
UsageStrip.show_details(self)	64	Open the per-generation token table and copy/export controls.

api_tester/ai/validation.py

Function / method signature	Line	Role / reference
has_variable(value: Any)	30	Internal formatting/check helper; see owning module in Chapter 3.
PlanIssue.as_text(self)	50	Internal formatting/check helper; see owning module in Chapter 3.
ValidationResult.ok(self)	60	Report whether this result has outstanding validation issues.
_options(values: Any, limit: int=30)	64	Internal formatting/check helper; see owning module in Chapter 3.
_canonical(value: str, allowed: Any)	70	Internal formatting/check helper; see owning module in Chapter 3.
validate_request_plan(plan: RequestPlan, index: CatalogIndex)	75	Validate/canonicalize request fields against catalog.
_coerce_scalar(value: str, type_name: str)	208	Normalise a scalar parameter value for its C# type; ``None`` if it cannot be read.
_split(value: str, operator: str)	239	Internal formatting/check helper; see owning module in Chapter 3.
_validate_filters(filters: list[FilterSpec], schema: FilterSchema None, issues: list[PlanIssue])	245	Check filterable fields, operators and value counts.
_validate_sort(sort: list[SortSpec], schema: FilterSchema None, issues: list[PlanIssue])	314	Check sortable properties and direction.
_validate_json_patch(payload: Any, issues: list[PlanIssue], assumptions: list[str])	340	Check RFC 6902 operation-array shape.
_validate_payload(payload: Any, schema: dict[str, Any] None, issues: list[PlanIssue], warnings: list[str])	380	Check/normalize available payload field metadata.

api_tester/ai/workflow_plan.py

Function / method signature	Line	Role / reference
WorkflowPlan.__post_init__(self)	107	Normalize/check dataclass fields after construction.
WorkflowPlan.to_json(self)	114	Serialize a typed draft into its machine-readable plan representation.

Function / method signature	Line	Role / reference
<code>WorkflowPlanner.__init__(self, *base_args: Any, mode: str, columns: tuple[str, ...]=(), **base_kwargs: Any)</code>	140	Initialize this component's state and dependencies.
<code>WorkflowPlanner.system_prompt(self)</code>	157	Choose instructions for the workflow.
<code>WorkflowPlanner.plan(self, prompt: str)</code>	164	Coordinate a planning operation and return outcome, issues and usage.
<code>WorkflowPlanner.output_token_limit(self)</code>	169	Return the fixed per-call workflow generation ceiling.
<code>WorkflowPlanner.response_schema(self, candidate_ids: list[str])</code>	172	Build the model's expected JSON contract.
<code>WorkflowPlanner.parse_plan(self, value: Any)</code>	256	Parse the selected workflow's model output.
<code>WorkflowPlanner.validate_plan(self, plan: RequestPlan)</code>	290	Run deterministic checks against catalog/workflow rules.
<code>validate_workflow_plan(plan: WorkflowPlan, index: CatalogIndex, columns: tuple[str, ...]=())</code>	299	Revalidate a persisted workflow draft against the current catalog.
<code>_validate_workflow_mappings(plan: WorkflowPlan, index: CatalogIndex, columns: tuple[str, ...], endpoint: Endpoint)</code>	389	Check CSV columns, destinations and transformations.
<code>_require_allowed_keys(value: dict[str, Any], required: set[str], allowed: set[str], label: str)</code>	485	Reject unsupported object keys.
<code>_parse_mapping(value: Any, position: int)</code>	499	Reject unsupported CSV target/transform shapes.
<code>_validate_transform_options(kind: str, options: dict[str, Any], path: str)</code>	533	Internal formatting/check helper; see owning module in Chapter 3.
<code>_parse_stage(value: Any, position: int)</code>	551	Parse finite, exact-type, representable load-stage values.
<code>_parse_threshold(value: Any, position: int)</code>	590	Parse supported bounded load threshold.
<code>_require_exact_keys(value: dict[str, Any], expected: set[str], path: str)</code>	605	Reject unexpected/missing keys at a strict workflow boundary.
<code>_strict_string(value: Any, path: str, *, strip: bool=True)</code>	614	Require the declared text type.
<code>_strict_integer(value: Any, path: str)</code>	620	Reject booleans/strings/fractions where an integer is required.
<code>_strict_number(value: Any, path: str)</code>	626	Require finite numeric values instead of coercing arbitrary strings.
<code>_ensure_finite_json(value: Any, path: str)</code>	638	Reject non-finite values throughout nested input.
<code>_is_sensitive_name(name: str)</code>	658	Internal formatting/check helper; see owning module in Chapter 3.
<code>_is_file_path_name(name: str)</code>	663	Internal formatting/check helper; see owning module in Chapter 3.
<code>_looks_like_local_path(value: str)</code>	668	Internal formatting/check helper; see owning module in Chapter 3.
<code>_contains_local_path(value: Any)</code>	675	Internal formatting/check helper; see owning module in Chapter 3.
<code>_has_decimal_places(value: float, places: int)</code>	685	Internal formatting/check helper; see owning module in Chapter 3.
<code>_validate_load_sections(plan: WorkflowPlan)</code>	693	Check workload stages, totals and threshold rules.

Function / method signature	Line	Role / reference
<code>_with_mapped_path_placeholders(plan: WorkflowPlan)</code>	788	Internal formatting/check helper; see owning module in Chapter 3.
<code>_unresolved_value_issues(plan: WorkflowPlan, *, allow_csv_path: bool)</code>	806	Reject placeholders not representable by native workflow.
<code>_local_path_value_issues(plan: WorkflowPlan, endpoint: Endpoint)</code>	854	Reject filesystem-path values in inappropriate AI targets.
<code>_remove_mapped_payload_warnings(warnings: tuple[str, ...], mappings: tuple[ColumnMapping, ...])</code>	902	Internal formatting/check helper; see owning module in Chapter 3.
<code>to_workflow_request(plan: WorkflowPlan, index: CatalogIndex)</code>	928	Convert a validated workflow draft to an Explorer request without CSV markers.
<code>render_workflow_outcome(outcome: PlanOutcome, index: CatalogIndex)</code>	947	Render a workflow result as escaped HTML; this function never executes it.

api_tester/ai/providers/fake.py

Function / method signature	Line	Role / reference
<code>FakeProvider.__init__(self, script: list[LlmResponse Exception] None=None)</code>	13	Initialize this component's state and dependencies.
<code>FakeProvider.complete(self, messages, **kwargs)</code>	17	Execute or wrap one provider model call using the common contract.
<code>FakeProvider.check_connection(self, model: str, timeout_seconds: float=10.0)</code>	26	Check provider/model availability and return an explicit report.

api_tester/ai/providers/hosted.py

Function / method signature	Line	Role / reference
<code>HostedProvider.__init__(self, name: str, endpoint: str, api_key: str, *, api_version: str='2024-10-21', session: requests.Session None...)</code>	21	Initialize this component's state and dependencies.
<code>HostedProvider.complete(self, messages: list[ChatMessage], *, model: str, tools: list[ToolSpec] None=None, response_schema: dict[str, ...])</code>	38	Execute or wrap one provider model call using the common contract.
<code>HostedProvider._post(self, url: str, body: dict[str, Any], timeout: float)</code>	169	Send HTTP request and convert provider failures to explicit errors.
<code>HostedProvider.for_connection_check(self)</code>	203	Clone adapter with isolated probe-only behavior.
<code>HostedProvider.check_connection(self, model: str, timeout_seconds: float=10.0)</code>	209	Check provider/model availability and return an explicit report.

api_tester/ai/providers/ollama.py

Function / method signature	Line	Role / reference
<code>parse_json_text(text: str)</code>	35	Parses model output as JSON, tolerating a surrounding Markdown fence.
<code>OllamaProvider.__init__(self, endpoint: str=DEFAULT_ENDPOINT, *, think: bool=False, keep_alive: str='30m', context_window: int None=None...)</code>	47	Initialize this component's state and dependencies.
<code>OllamaProvider.build_body(self, messages: list[ChatMessage], *, model: str, tools: list[ToolSpec] None, response_schema: dict[str, Any] ...)</code>	63	Construct the native Ollama payload/schema/options.
<code>OllamaProvider._message(message: ChatMessage)</code>	100	Internal formatting/check helper; see owning module in Chapter 3.

Function / method signature	Line	Role / reference
<code>OllamaProvider.complete(self, messages: list[ChatMessage], *, model: str, tools: list[ToolSpec] None=None, response_schema: dict[str, ...])</code>	111	Execute or wrap one provider model call using the common contract.
<code>OllamaProvider.parse_response(data: dict[str, Any], *, expect_json: bool, duration_ms: int=0)</code>	139	Normalize provider content, tool data and reported metrics.
<code>OllamaProvider.list_models(self, timeout_seconds: float=10.0)</code>	172	Read and validate Ollama's documented available-model list.
<code>OllamaProvider.check_connection(self, model: str, timeout_seconds: float=10.0)</code>	194	Check provider/model availability and return an explicit report.
<code>OllamaProvider._post(self, path: str, body: dict[str, Any], timeout: float, model: str)</code>	206	Send HTTP request and convert provider failures to explicit errors.
<code>_arguments(value: Any)</code>	229	Internal formatting/check helper; see owning module in Chapter 3.
<code>_error_detail(response: requests.Response)</code>	241	Internal formatting/check helper; see owning module in Chapter 3.

api_tester/runner.py

Function / method signature	Line	Role / reference
<code>CaseResult.baseline_expected_text(self)</code>	72	Internal formatting/check helper; see owning module in Chapter 3.
<code>CaseResult.baseline_expected_text(self, value: str)</code>	76	Internal formatting/check helper; see owning module in Chapter 3.
<code>CaseResult.baseline_actual_text(self)</code>	80	Internal formatting/check helper; see owning module in Chapter 3.
<code>CaseResult.baseline_actual_text(self, value: str)</code>	84	Internal formatting/check helper; see owning module in Chapter 3.
<code>CaseResult.outcome(self)</code>	88	Internal formatting/check helper; see owning module in Chapter 3.
<code>CaseResult.failed_assertions(self)</code>	96	Internal formatting/check helper; see owning module in Chapter 3.
<code>SuiteResult.total(self)</code>	108	Internal formatting/check helper; see owning module in Chapter 3.
<code>SuiteResult.executed(self)</code>	112	Internal formatting/check helper; see owning module in Chapter 3.
<code>SuiteResult.passed(self)</code>	116	Internal formatting/check helper; see owning module in Chapter 3.
<code>SuiteResult.failed(self)</code>	120	Internal formatting/check helper; see owning module in Chapter 3.
<code>SuiteResult.errorred(self)</code>	124	Internal formatting/check helper; see owning module in Chapter 3.
<code>SuiteResult.skipped(self)</code>	128	Internal formatting/check helper; see owning module in Chapter 3.
<code>SuiteResult.duration_ms(self)</code>	132	Internal formatting/check helper; see owning module in Chapter 3.
<code>SuiteResult.wall_duration_ms(self)</code>	136	Wall-clock span of the run, including between-case overhead.

Function / method signature	Line	Role / reference
<code>SuiteResult.pass_rate(self)</code>	146	Internal formatting/check helper; see owning module in Chapter 3.
<code>SuiteResult.slowest(self, count: int=5)</code>	150	Internal formatting/check helper; see owning module in Chapter 3.
<code>SuiteResult.cleanup_failed(self)</code>	154	Internal formatting/check helper; see owning module in Chapter 3.
<code>SuiteResult.cleanup_errored(self)</code>	162	Internal formatting/check helper; see owning module in Chapter 3.
<code>_decode(body: str)</code>	168	Internal formatting/check helper; see owning module in Chapter 3.
<code>_baseline_outcome(case: TestCase, document: Any, valid_json: bool)</code>	178	Internal formatting/check helper; see owning module in Chapter 3.
<code>run_case(case: TestCase, endpoint: Endpoint, base_url: str, access_token: str, api_key: str, variables: dict[str, str], t...)</code>	203	Executes one case and evaluates its assertions, updating ``variables``.
<code>_phase_order(suite: TestSuite, phase: str)</code>	330	Return a stable dependency order, reversed for teardown.
<code>_case_result(case: TestCase, endpoint: Endpoint None, *, skipped: bool=False, skip_reason: str='', error: str='')</code>	353	Internal formatting/check helper; see owning module in Chapter 3.
<code>run_suite(suite: TestSuite, endpoints: dict[str, Endpoint], base_urls: dict[str, str], access_token: str, api_key: str, ti...)</code>	377	Run dependency-ordered normal cases, then reverse-ordered cleanup cases.
<code>append_suite_history(target: 'WorkspaceStore Path str', result: SuiteResult)</code>	481	Records one suite run in the workspace database.
<code>export_report(result: SuiteResult, path: Path)</code>	502	Writes a full JSON report of a suite run.

api_tester/main.py

Function / method signature	Line	Role / reference
<code>MainWindow._check_ai_connection(self)</code>	1208	Internal formatting/check helper; see owning module in Chapter 3.
<code>MainWindow._show_ask_ai(self)</code>	2770	Internal formatting/check helper; see owning module in Chapter 3.
<code>MainWindow._ai_execution_finished(self, snapshot: dict)</code>	2799	Internal formatting/check helper; see owning module in Chapter 3.
<code>MainWindow.open_ai_request(self, request)</code>	2860	Opens an Ask AI plan in API Explorer as an editable, unsent draft.
<code>MainWindow.open_ai_suite(self, suite: TestSuite)</code>	2879	Loads an Ask AI suite into Test Suites, unsaved and not yet run.
<code>MainWindow.open_ai_data(self, plan: WorkflowPlan, request: ExplorerRequest, source: CsvImportSettings, columns: tuple[str, ...])</code>	2899	Hand off a CSV mapping draft; require native validation before Run.
<code>MainWindow.open_ai_load(self, plan: WorkflowPlan, request: ExplorerRequest)</code>	2938	Hand off editable load stages without granting load/write consent.
<code>MainWindow._validate_ai_workflow(self, plan: WorkflowPlan, columns: tuple[str, ...])=()</code>	2975	Internal formatting/check helper; see owning module in Chapter 3.

api_tester/data_runner/ui.py

Function / method signature	Line	Role / reference
DataRunnerTab.load_ai_draft(self, endpoint: Endpoint, values: dict[str, str], payload: Any, source: CsvImportSettings, columns: tuple[str, ...])	1154	Import a reviewed CSV mapping without validating or executing rows.

api_tester/load_testing/ui.py

Function / method signature	Line	Role / reference
LoadTestingTab.load_ai_draft(self, endpoint: Endpoint, values: dict[str, str], payload: Any, *, name: str, stages: tuple[LoadStage, ...], thr...)	454	Open a load configuration without granting permissions or starting it.

api_tester/data_runner/csv_source.py

Function / method signature	Line	Role / reference
CsvSource.read_columns(self)	137	Read only the header for AI mapping; never return row data.